

PL/SQL Exercises - Solutions

Schema Used

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    DOB DATE,  
    Balance NUMBER,  
    LastModified DATE  
);  
  
CREATE TABLE Accounts (  
    AccountID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    AccountType VARCHAR2(20),  
    Balance NUMBER,  
    LastModified DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)  
);
```

```

CREATE TABLE Transactions (
    TransactionID NUMBER PRIMARY KEY,
    AccountID NUMBER,
    TransactionDate DATE,
    Amount NUMBER,
    TransactionType VARCHAR2(10),
    FOREIGNKEY (AccountID) REFERENCES Accounts (AccountID)
);

CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    LoanAmount NUMBER,
    InterestRate NUMBER,
    StartDate DATE,
    EndDate DATE,
    FOREIGNKEY (CustomerID) REFERENCES Customers (CustomerID)
);

CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    Position VARCHAR2(50),
    Salary NUMBER,
    Department VARCHAR2(50),
    HireDate DATE
);

```

Also added an AuditLog table for Exercise 5 (Scenario 2):

```

CREATE TABLE AuditLog (
    LogID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    TransactionID NUMBER,
    LogMessage VARCHAR2(200),
    LogDate DATE DEFAULT SYSDATE
);

```

Exercise 1: Control Structures

Scenario 1 - Loan interest discount for customers above 60

```

DECLARE
    v_age NUMBER;
BEGIN
    FOR cust IN (SELECT CustomerID, DOB FROM Customers) LOOP

```

```

v_age:=TRUNC (MONTHS_BETWEEN (SYSDATE, cust.DOB) /12) ;

IF v_age>60 THEN UPDATE
    Loans
    SET InterestRate=InterestRate-(InterestRate*0.01)
    WHERE CustomerID=cust.CustomerID;
END IF;
END LOOP;

COMMIT;
END;
/

```

Scenario 2 - Set IsVIP flag for balance over \$10,000

First need to add the column since it's not in the schema:

```

ALTER TABLE Customers ADD IsVIP VARCHAR2 (5) DEFAULT 'FALSE';

BEGIN
    FOR cust IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF cust.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = cust.CustomerID;
        END IF;
    END LOOP;

    COMMIT;
END;
/

```

Scenario 3 - Reminder for loans due in next 30 days

```

BEGIN
    FOR loan IN (
        SELECT l.LoanID, l.CustomerID, l.EndDate, c.Name
        FROM Loans l
        JOIN Customers c ON c.CustomerID = l.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Reminder: Loan' || loan.LoanID ||
                               ' for' || loan.Name ||
                               ' is due on' || TO_CHAR(loan.EndDate, 'DD-MON-YYYY'));
    END LOOP;
END;
/

```

Exercise 2: Error Handling

Scenario 1 - SafeTransferFunds procedure

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds ( p_from_account IN
    NUMBER,
    p_to_account          IN NUMBER,
    p_amount              IN NUMBER
) AS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient funds in account' || p_from_acco
    END IF;

    UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID = p_from_account;
    UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID = p_to_account;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer successful');

EXCEPTION
    WHEN NO_DATA_FOUND
    THEN ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: Account not found');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Scenario 2 - UpdateSalary procedure

```
CREATE OR REPLACE PROCEDURE UpdateSalary (
    p_employee_id IN NUMBER,
    p_percent      IN NUMBER
) AS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM Employees WHERE EmployeeID = p_employee_id;
```

```

    IF v_count=0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'EmployeeIDdoesnotexist');
    ENDIF;

    UPDATE Employees
    SET Salary=Salary+(Salary*p_percent/100)
    WHERE EmployeeID=p_employee_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salaryupdatedsuccessfully');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

Scenario 3 - AddNewCustomer procedure

```

CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_customer_id IN NUMBER, p_name
                                IN VARCHAR2,
    p_dob          IN DATE,
    p_balance      IN NUMBER
) AS
BEGIN
    N
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified) VALUES (p_customer_id,
    p_name, p_dob, p_balance, SYSDATE);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Customer added successfully');

EXCEPTION
    WHEN DUP_VAL_ON_INDEX
        THEN ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Customer ID' || p_customer_id || 'already exists
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

Exercise 3: Stored Procedures

Scenario 1 - ProcessMonthlyInterest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01),
        LastModified = SYSDATE
    WHERE AccountType = 'Savings';

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly interest applied to all savings accounts');
END;
/
```

Scenario 2 - UpdateEmployeeBonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus ( p_department IN
    VARCHAR2,
    p_bonus_pct IN NUMBER
) AS
BEGIN
    N
    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus_pct / 100)
    WHERE Department = p_department;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Bonus applied to ' || p_department || ' department');
END;
/
```

Scenario 3 - TransferFunds

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
) AS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = p_from_account;

    IF v_balance >= p_amount THEN
        UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID = p_from_account;
        UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID = p_to_account;
    END IF;
END;
```

```

        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Transfer of ' || p_amount || ' completed');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Transfer failed: insufficient balance');
    ENDIF;
END;
/

```

Exercise 4: Functions

Scenario 1 - CalculateAge

```

CREATE OR REPLACE FUNCTION CalculateAge (
    p_dob IN DATE
) RETURN NUMBER AS
    v_age NUMBER;
BEGIN
    v_age := TRUNC(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
    RETURN v_age;
END;
/

```

Usage:

```

SELECT CalculateAge(DOB) AS Age FROM Customers WHERE CustomerID = 1;

```

Scenario 2 - CalculateMonthlyInstallment

```

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment ( p_loan_amount
    IN NUMBER,
    p_rate          IN NUMBER,      -- annual interest rate, e.g. 5 for 5%
    p_years         IN NUMBER
) RETURN NUMBER AS
    v_monthly_rate NUMBER;
    v_months
        NUMBER
    ;
    v_emi
        NUMBER;
BEGIN
    v_monthly_rate := (p_rate / 100) / 12; v_months := p_years
    * 12;

    -- standard EMI formula
    v_emi := (p_loan_amount * v_monthly_rate * POWER(1 + v_monthly_rate, v_months))
    / (POWER(1 + v_monthly_rate, v_months) - 1);

```

```
RETURN ROUND(v_emi,2);
```

```
END;  
/
```

Scenario 3 - HasSufficientBalance

```
CREATE OR REPLACE FUNCTION HasSufficientBalance ( p_account_id IN  
    NUMBER,  
    p_amount          IN NUMBER  
) RETURN BOOLEAN AS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = p_account_id;  
  
    IF v_balance >= p_amount THEN RETURN  
        TRUE;  
    ELSE  
        RETURN FALSE;  
    END IF;  
END;  
/
```

Note: since this returns BOOLEAN, it can't be called directly in a plain SELECT. It's meant to be used inside another PL/SQL block, like this:

```
DECLARE  
    v_result BOOLEAN;  
BEGIN  
    v_result := HasSufficientBalance(1, 500);  
    IF v_result THEN  
        DBMS_OUTPUT.PUT_LINE('Sufficient balance');  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Not enough balance');  
    END IF;  
END;  
/
```

Exercise 5: Triggers

Scenario 1 - UpdateCustomerLastModified

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified  
BEFORE UPDATE ON Customers  
FOR EACH ROW  
BEGIN  
    :NEW.LastModified := SYSDATE;
```

```
END;  
/
```

Scenario 2 - LogTransaction

```
CREATE OR REPLACE TRIGGER LogTransaction AFTER  
INSERT ON Transactions  
FOR EACH ROW  
BEGIN  
    INSERT INTO AuditLog (TransactionID, LogMessage, LogDate) VALUES  
    (:NEW.TransactionID,  
        'New ' || :NEW.TransactionType || ' of ' || :NEW.Amount || ' recorded', SYSDATE);  
END;  
/
```

Scenario 3 - CheckTransactionRules

```
CREATE OR REPLACE TRIGGER CheckTransactionRules BEFORE  
INSERT ON Transactions  
FOR EACH ROW  
DECLARE  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = :NEW.AccountID;  
  
    IF :NEW.TransactionType = 'Withdrawal' AND :NEW.Amount > v_balance THEN  
        RAISE_APPLICATION_ERROR(-20003, 'Withdrawal amount exceeds account balance');  
    END IF;  
  
    IF :NEW.TransactionType = 'Deposit' AND :NEW.Amount <= 0 THEN  
        RAISE_APPLICATION_ERROR(-20004, 'Deposit amount must be positive');  
    END IF;  
END;  
/
```

Exercise 6: Cursors

Scenario 1 - GenerateMonthlyStatements

```
DECLARE  
    CURSOR c_statements IS  
        SELECT t.TransactionID, t.AccountID, t.Amount, t.TransactionType, t.Transaction
```

```

        FROM Transactions t
        WHERE TRUNC(t.TransactionDate, 'MM') = TRUNC(SYSDATE, 'MM');
BEGIN
N
    FOR rec IN c_statements LOOP
        DBMS_OUTPUT.PUT_LINE('Account ' || rec.AccountID || ' - ' ||
                               rec.TransactionType || ' of ' || rec.Amount ||
                               ' on ' || TO_CHAR(rec.TransactionDate, 'DD-MON-YYYY'));
    END LOOP;
END;
/

```

Scenario 2 - ApplyAnnualFee

```

DECLARE
    CURSOR c_accounts IS SELECT AccountID FROM Accounts;
    v_fee NUMBER := 50; -- flat annual fee
BEGIN
    FOR rec IN c_accounts LOOP
        UPDATE Accounts
        SET Balance = Balance - v_fee
        WHERE AccountID = rec.AccountID;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Annual fee applied to all accounts');
END;
/

```

Scenario 3 - UpdateLoanInterestRates

```

DECLARE
    CURSOR c_loans IS SELECT LoanID FROM Loans;
    v_new_rate NUMBER := 6; -- new policy rate
BEGIN
    FOR rec IN c_loans LOOP
        UPDATE Loans
        SET InterestRate = v_new_rate
        WHERE LoanID = rec.LoanID;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Loan interest rates updated for all loans');
END;
/

```

Exercise 7: Packages

Scenario 1 - CustomerManagement package

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_balance NUMBER);
    PROCEDURE UpdateCustomerDetails(p_id NUMBER, p_name VARCHAR2);
    FUNCTION GetCustomerBalance(p_id NUMBER) RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_balance NUMBER) IS BEGIN
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified) VALUES (p_id,
            p_name, p_dob, p_balance, SYSDATE);
        COMMIT;
    END AddCustomer;

    PROCEDURE UpdateCustomerDetails(p_id NUMBER, p_name VARCHAR2) IS BEGIN
        UPDATE Customers
        SET Name = p_name, LastModified = SYSDATE WHERE
        CustomerID = p_id;
        COMMIT;
    END UpdateCustomerDetails;

    FUNCTION GetCustomerBalance(p_id NUMBER) RETURN NUMBER IS
        v_balance NUMBER;
    BEGIN
        SELECT Balance INTO v_balance FROM Customers WHERE CustomerID = p_id;
        RETURN v_balance;
    END GetCustomerBalance;

END CustomerManagement;
/
```

Scenario 2 - EmployeeManagement package

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_position VARCHAR2,
        p_salary NUMBER, p_dept VARCHAR2);
    PROCEDURE UpdateEmployeeDetails(p_id NUMBER, p_position VARCHAR2, p_dept VARCHAR2);
    FUNCTION CalculateAnnualSalary(p_id NUMBER) RETURN NUMBER;
END EmployeeManagement;
/
```

```
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
```

```
PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_position VARCHAR2,  
                        p_salary NUMBER, p_dept VARCHAR2) IS  
BEGIN  
    INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate VALUES (p_id, p_name,  
    p_position, p_salary, p_dept, SYSDATE);  
    COMMIT;  
END HireEmployee;  
  
PROCEDURE UpdateEmployeeDetails(p_id NUMBER, p_position VARCHAR2, p_dept VARCHAR2) BEGIN  
    UPDATE Employees  
    SET Position = p_position, Department = p_dept  
    WHERE EmployeeID = p_id;  
    COMMIT;  
END UpdateEmployeeDetails;  
  
FUNCTION CalculateAnnualSalary(p_id NUMBER) RETURN NUMBER IS  
    v_salary NUMBER;  
BEGIN  
    SELECT Salary INTO v_salary FROM Employees WHERE EmployeeID = p_id;  
    RETURN v_salary * 12;  
END CalculateAnnualSalary;  
  
END EmployeeManagement;  
/
```

Scenario 3 - AccountOperations package

```
CREATE OR REPLACE PACKAGE AccountOperations AS
```

```
    PROCEDURE OpenAccount(p_id NUMBER, p_customer_id NUMBER, p_type VARCHAR2, p_balance  
    PROCEDURE CloseAccount(p_id NUMBER);  
    FUNCTION GetTotalBalance(p_customer_id NUMBER) RETURN NUMBER;  
END AccountOperations;  
/
```

```
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
```

```
    PROCEDURE OpenAccount(p_id NUMBER, p_customer_id NUMBER, p_type VARCHAR2, p_balance BEGIN  
        INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified VALUES (p_id,  
        p_customer_id, p_type, p_balance, SYSDATE);  
        COMMIT;  
    END OpenAccount;
```

```
PROCEDURE CloseAccount(p_id NUMBER) IS BEGIN  
    DELETE FROM Accounts WHERE AccountID = p_id;  
    COMMIT;  
END CloseAccount;  
  
FUNCTION GetTotalBalance(p_customer_id NUMBER) RETURN NUMBER IS  
    v_total NUMBER;  
BEGIN  
    SELECT SUM(Balance) INTO v_total FROM Accounts WHERE CustomerID = p_customer_id  
    RETURN NVL(v_total, 0);  
END GetTotalBalance;
```

```
END AccountOperations;
```

```
/
```